# Pre-Reading: What Is a Stack? What Is a Queue?

**Read this before starting P2.** It takes about 10 minutes.

This document explains two data structures you'll use in FIT1061: **stacks** and **queues**. You don't need to implement them; we give them to you. But you need to understand how they behave, because the choice between a stack and a queue is what makes the difference between two very different search algorithms.

---

## The core idea: ordering

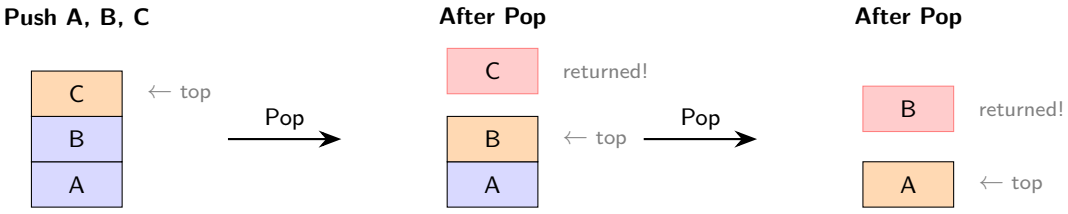
Both stacks and queues are collections that store items. The difference is **which item comes out first** when you remove one.

Think of a stack of plates and a queue of people.

---

## Stack: Last In, First Out (LIFO)

A **stack** works like a stack of plates in a cafeteria. You add plates to the top and take plates from the top. The last plate you put on is the first one you take off.




**Key insight:** The last item pushed (C) is the first item popped. This is **Last In, First Out (LIFO)**.

**Operations:**

- `push(item)`: add an item to the **top**
- `pop()`: remove and return the item from the **top**
- `is_empty()`: check if the stack has no items

**Real-world analogy:** Undo in a text editor. Each action goes on top of the stack. "Undo" pops the most recent action.

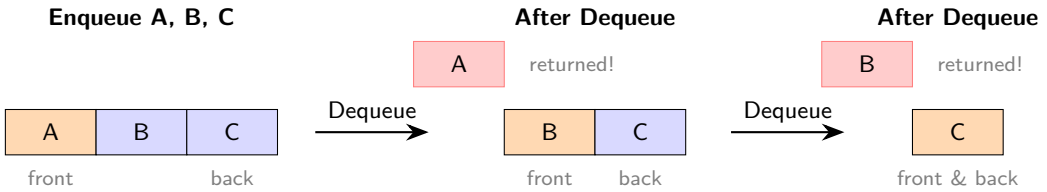
**In Python,** a list can act as a stack:

```
stack = []
stack.append("A")   # push
stack.append("B")   # push
top = stack.pop()   # pop → "B"
```

---

## Queue: First In, First Out (FIFO)

A **queue** works like a queue of people at a ticket counter. The first person to join the queue is the first person served. New people join at the back.




**Key insight:** The first item enqueued (A) is the first item dequeued. This is **First In, First Out (FIFO)**.

**Operations:**

- `enqueue(item)`: add an item to the **back**
- `dequeue()`: remove and return the item from the **front**
- `is_empty()`: check if the queue has no items

**Real-world analogy:** A printer queue. Documents print in the order they were sent.

**In Python,** you *can* use a list as a queue (`list.pop(0)`), but it's slow. We provide a `Queue` class that handles this efficiently.

---

## Quick self-test

Before moving on, make sure you can answer these:

1. You push A, B, C onto a stack, then pop twice. What do you get?
2. You enqueue A, B, C into a queue, then dequeue twice. What do you get?
3. Which data structure would you use if you want to process items in the order they arrived?

**Answers:** (1) C, then B. (2) A, then B. (3) A queue.

---

> **Want to learn more?** Stacks and queues are fundamental data structures in computer science. You'll study them in much more detail in **FIT1008 Introduction to Computer Science**, including how they are implemented and their performance characteristics. For FIT1061, you just need to understand how they *behave*: which item comes out first.

---